```
# used with time series data
```

```
#Error gradient=> similar to=>MSE
```

```
import numpy as np
```

```
def generate_sequence(length, noise=0.1):
  return np.array([i + np.random.normal(0, noise) for i in range(length)])
```

```
def create_dataset(seq, n_steps):
  X, y = [], []
  for i in range(len(seq) - n_steps):
    X.append(seq[i: i+n_steps])
    y.append(seq[i + n_steps])
  return np.array(X), np.array(y)
```

```
generate_sequence(10, 0.1)
```

```
array([0.04717444, 1.13442087, 2.04171133, 3.15818947, 3.85451274,
       5.12838365, 6.08877639, 6.75114621, 8.01860227, 9.03322652])
```

```
create_dataset(generate_sequence(10), 5)
```

```
(array([[-0.08758877,  1.19404542,  1.88184083,  2.86636893,  4.01521953],
        [ 1.19404542,  1.88184083,  2.86636893,  4.01521953,  5.1026482 ],
        [ 1.88184083,  2.86636893,  4.01521953,  5.1026482 ,  6.06585537],
        [ 2.86636893,  4.01521953,  5.1026482 ,  6.06585537,  7.20334692],
        [ 4.01521953,  5.1026482 ,  6.06585537,  7.20334692,  8.12949915]]),
 array([5.1026482 , 6.06585537, 7.20334692, 8.12949915, 8.93797093]))
```

```
np.random.seed(42)
```

```
sequence = generate_sequence(100, noise=0.5)
```

```
n_steps = 3
X, y = create_dataset(sequence, n_steps)
X = X.reshape(X.shape[0], X.shape[1], 1)
```

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.regularizers import l2
from tensorflow.keras.layers import SimpleRNN, Dense
```

```
model = Sequential([
    SimpleRNN(50, activation="relu", input_shape=(n_steps, 1),
              kernel_regularizer=l2(0.01), recurrent_regularizer=l2(0.001),
               ),
               Dense(1)
    ]

)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
  super().__init__(**kwargs)
```

```
optimizer = tf.keras.optimizers.Adam(clipvalue=1)
model.compile(optimizer = optimizer, loss='mse')
```

```
model.fit(X, y, epochs=100, verbose=0)
```

```
<keras.src.callbacks.history.History at 0x7b08e4300b00>
```

```
x_input = np.array([97, 98, 99,])
x_input = x_input.reshape(1, n_steps, 1)
y_hat = model.predict(x_input, verbose=0)
print(f"Predicted next value: \t{y_hat[0][0]:.4f}")
print(f"Actual next value: \t~100")
```

```
Predicted next value:   100.4879
Actual next value:      ~100
```

Example 2

```
def generate_temperatue_data(seq_length, num_samples):
  np.random.seed(42)
  base_temp = 20      # average temperature in C
  temp_variation = 5  # max daily temp variation

  data = base_temp + temp_variation * np.sin(np.linspace(0,100, num_samples))

  X = []
  y = []
```

```
  for i in range(len(data) - seq_length):
    X.append(data[i: i+seq_length])
    y.append(data[i :  i+seq_length]) # Corrected slice end here as well, assuming it should be length

  return np.array(X), np.array(y)
```

```
generate_temperatue_data(5, 10)
```

```
(array([[20.        , 15.03333479, 18.85488617, 24.70264788, 22.22935585],
        [15.03333479, 18.85488617, 24.70264788, 22.22935585, 15.81135218],
        [18.85488617, 24.70264788, 22.22935585, 15.81135218, 16.80490993],
        [24.70264788, 22.22935585, 15.81135218, 16.80490993, 23.45198818],
        [22.22935585, 15.81135218, 16.80490993, 23.45198818, 23.99098012]]),
 array([[20.        , 15.03333479, 18.85488617, 24.70264788, 22.22935585],
        [15.03333479, 18.85488617, 24.70264788, 22.22935585, 15.81135218],
        [18.85488617, 24.70264788, 22.22935585, 15.81135218, 16.80490993],
        [24.70264788, 22.22935585, 15.81135218, 16.80490993, 23.45198818],
        [22.22935585, 15.81135218, 16.80490993, 23.45198818, 23.99098012]]))
```

```
seq_length = 30
num_samples = 365  # Simulating a year of data
X, y = generate_temperatue_data(seq_length, num_samples)
X = X[..., np.newaxis]  # Add an extra dimension for compatibility with RNN input
```

```
X
```

```
array([[[20.        ],
        [21.35641262],
        [22.61109365],
        ...,
        [24.53139755],
        [24.93479605],
        [24.96808251]],

       [[21.35641262],
        [22.61109365],
        [23.66994141],
        ...,
        [24.93479605],
        [24.96808251],
        [24.6287604 ]],

       [[22.61109365],
        [23.66994141],
        [24.45354182],
        ...,
        [24.96808251],
        [24.6287604 ],
        [23.94227907]],

       ...,

       [[19.4878784 ],
        [18.15780411],
        [16.96589538],
        ...,
        [15.27597098],
        [15.00872053],
        [15.11581842]],

       [[18.15780411],
        [16.96589538],
        [16.00154595],
        ...,
        [15.00872053],
        [15.11581842],
        [15.58923225]],

       [[16.96589538],
        [16.00154595],
        [15.33708247],
        ...,
        [15.11581842],
        [15.58923225],
        [16.39345575]]])
```

```
model = Sequential([
    SimpleRNN(50, activation='tanh', input_shape=(seq_length, 1)),
    Dense(1)
])

model.compile(optimizer='adam', loss='mse')
model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
  super().__init__(**kwargs)
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
simple_rnn_1 (SimpleRNN)	(None, 50)	2,600
dense_1 (Dense)	(None, 1)	51

Total params: 2,651 (10.36 KB)
Trainable params: 2,651 (10.36 KB)
Non-trainable params: 0 (0.00 B)

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

history = model.fit(X_train, y_train, epochs=20, batch_size=32, validation_data=(X_test, y_test))
```

```
Epoch 1/20
9/9 ━━━━━━━━━━━━━━━━━━━━ 4s 43ms/step - loss: 339.4597 - val_loss: 312.3916
Epoch 2/20
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 289.7376 - val_loss: 266.9579
Epoch 3/20
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 246.9014 - val_loss: 229.1696
Epoch 4/20
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 212.6566 - val_loss: 197.9190
Epoch 5/20
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - loss: 182.3172 - val_loss: 168.5829
Epoch 6/20
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - loss: 154.2821 - val_loss: 141.7307
Epoch 7/20
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 128.9288 - val_loss: 117.8900
Epoch 8/20
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 108.2219 - val_loss: 101.5701
Epoch 9/20
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 95.1384 - val_loss: 91.1655
Epoch 10/20
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 86.4399 - val_loss: 83.7411
Epoch 11/20
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 79.7505 - val_loss: 77.6264
Epoch 12/20
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 74.0340 - val_loss: 72.1339
Epoch 13/20
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 68.8279 - val_loss: 67.0977
Epoch 14/20
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - loss: 63.9996 - val_loss: 62.4540
Epoch 15/20
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - loss: 59.5684 - val_loss: 58.1164
Epoch 16/20
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 55.4220 - val_loss: 54.0877
Epoch 17/20
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 51.5619 - val_loss: 50.3509
Epoch 18/20
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 47.9914 - val_loss: 46.8787
Epoch 19/20
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 44.6701 - val_loss: 43.6671
Epoch 20/20
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 41.6116 - val_loss: 40.6951
```

```
def predict_future(model, recent_data, days):
    predictions = []
    input_seq = recent_data[-seq_length:]

    for _ in range(days):
        pred = model.predict(input_seq[np.newaxis, ...])[0, 0]
        predictions.append(pred)
        input_seq = np.append(input_seq[1:], [[pred]], axis=0)

    return predictions
```

```
future_prediction = predict_future(model, X[-1], 7)
```

```
future_predictions = predict_future(model, X[-1], 7)
print("Predicted Temperatures for Next 7 Days:", future_predictions)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 158ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step
Predicted Temperatures for Next 7 Days: [np.float32(14.758574), np.float32(14.753251), np.float32(14.752961), np.float32(14.752956), np.float32(14.75295
```

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```